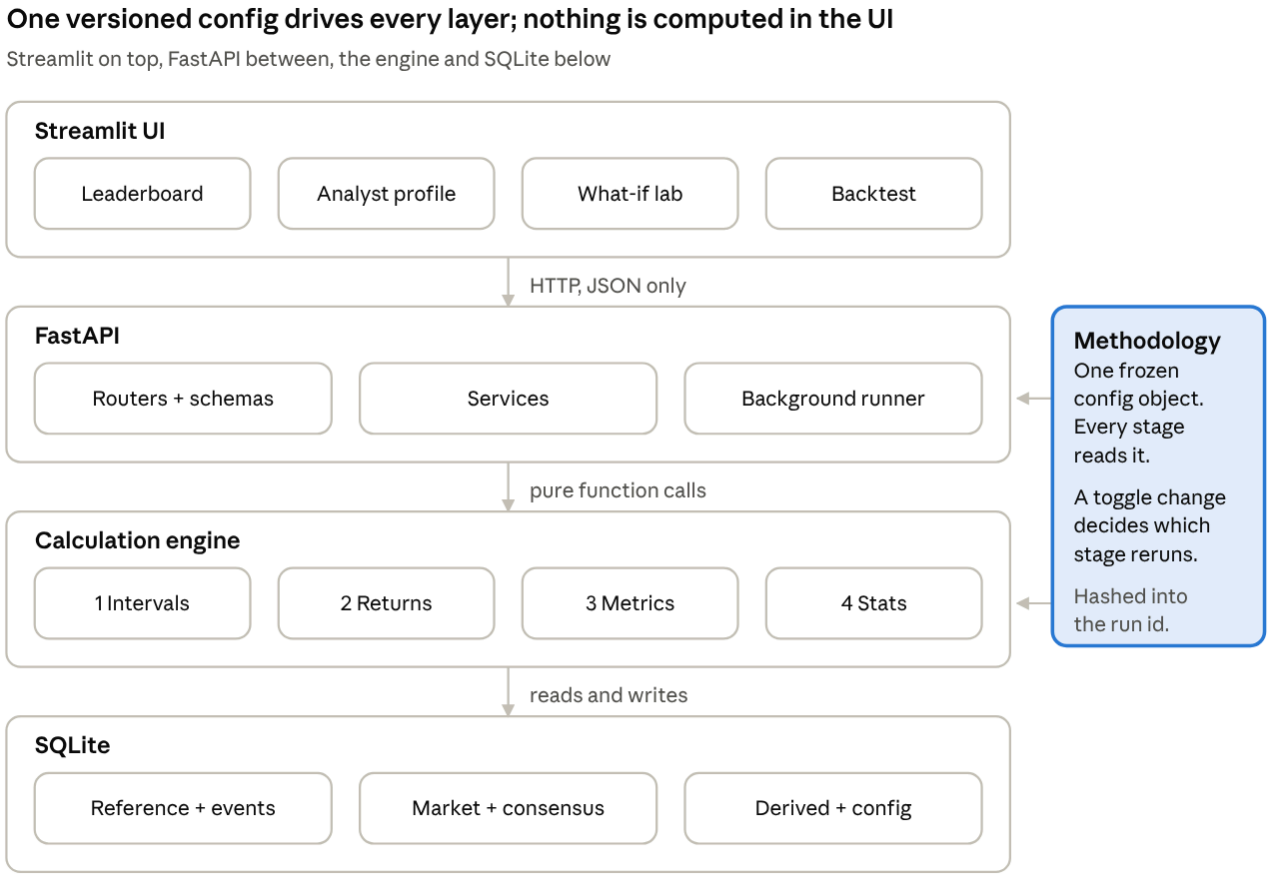
Analyst Rating Metrics Platform — Build Plan

Oct 6, 2026 · @sanjay

Nine layers, each one runnable end-to-end before the next begins: SQLite schema → synthetic data → calculation engine → FastAPI → Streamlit → what-if → backtest sandbox → hardening. Stack is Streamlit (UI), FastAPI (service), SQLite (store), with the methodology held as versioned config so every number can be reproduced.

Architecture at a glance



architecture · 4 layers, 1 config object

Streamlit holds no calculation logic; it only renders what the API returns. The engine is a pure function of the data plus the methodology object, which is what makes a what-if run cheap: change a toggle, and only the stages below the first affected one recompute.

Layer 0 — Foundations

Build the methodology object before anything else. Every later layer reads it, and retro-fitting it is the single most expensive mistake available here.

Repo layout

```
analyst-metrics/
  api/
    main.py            FastAPI app, router registration
    routers/           analysts, securities, ratings, metrics, scenarios,
backtest
    schemas/           Pydantic request/response models
    services/          thin orchestration over engine/
    deps.py            DB session, settings
  engine/
    intervals.py       rating events -> positions
    returns.py         total return, benchmark, excess
    metrics.py         the metric families
    stats.py           significance, shrinkage, peer normalization
    methodology.py     the config object + validation
    runner.py          orchestrates a full calculation run
  db/
    schema.sql         DDL
    models.py          SQLAlchemy models
    seed/              synthetic data generators
  ui/
    app.py             Streamlit entry
    pages/             one file per screen
    api_client.py      typed wrapper over requests
    components/        shared charts, filters, tables
  tests/
  data/analyst.db
```

Methodology config — the spine of the system

A single versioned object, stored as a row in `methodology_version` and loaded as a frozen Pydantic model. Nothing in the engine takes a hardcoded rule; every rule arrives through this object.

Parameter	Type	Default	What it controls
`entry_convention`	enum	`next_open`	`same_close`, `next_open`, `t_plus_2_open`

Parameter	Type	Default	What it controls
`expiry_policy`	enum	`hard_close`	`hard_close`, `carry_forward`, `decay`
`decay_days`	int	90	Taper length when policy is `decay`
`benchmark_mode`	enum	`sector`	`market`, `sector`, `beta_adjusted`, `custom`
`benchmark_id`	str	`IDX_BROAD`	Which series when mode is `market` or `custom`
`scoring_mode`	enum	`absolute`	`absolute` or `relative_to_consensus`
`position_weights`	dict	SB +1.0, B +0.5, H 0, S −0.5, SS −1.0	Rating → position size
`fixed_horizons_days`	list	[63, 126, 252]	Fixed windows reported alongside to-next-change
`min_ratings_for_rank`	int	20	Below this, analyst is scored but not ranked
`shrinkage_method`	enum	`empirical_bayes`	`none`, `empirical_bayes`
`peer_group_dims`	list	[sector, region, cap_bucket]	Normalization cohort
`transaction_cost_bps`	float	0.0	Applied per position change in simulation
`include_announcement_window`	bool	true	Compute CAR [−1,+1] separately
`return_basis`	enum	`total`	`total` or `price`

Rules that hold across every layer

- Store the native rating label forever; never overwrite it with the normalized value.
- Every derived table carries `methodology_version_id` and `run_id`. Results are never updated in place.
- All timestamps stored UTC; a `timezone` column on the exchange, converted only at display.
- Money stored as integer minor units or `NUMERIC`, never float.
- No calculation reads a price with a date later than the as-of date of the run.

Layer 1 — Data model

Five groups: reference, event, market, derived, config. Reference and event tables are the system of record; derived tables are disposable and rebuilt by a run.

Reference

Table	Key columns	Notes
`firm`	`firm_id`, `name`, `country`, `tier`	Research house
`analyst`	`analyst_id`, `full_name`, `seniority`, `start_year`	Identity-resolved person
`analyst_employment`	`analyst_id`, `firm_id`, `from_date`, `to_date`	Open `to_date` = current
`analyst_coverage`	`analyst_id`, `security_id`, `from_date`, `to_date`, `is_lead`	Coverage transfers create a new row
`security`	`security_id`, `primary_ticker`, `name`, `currency`, `exchange_id`, `sector_id`, `cap_bucket`, `listing_date`, `delisting_date`, `delisting_reason`	`delisting_reason` ∈ acquired, bankrupt, merged, voluntary
`security_identifier`	`security_id`, `id_type`, `id_value`, `from_date`, `to_date`	Point-in-time ticker/ISIN history
`sector`	`sector_id`, `name`, `parent_sector_id`	GICS-style two levels
`security_sector_history`	`security_id`, `sector_id`, `from_date`, `to_date`	PIT classification — reclassifications happen
`exchange`	`exchange_id`, `name`, `timezone`, `country`	
`trading_calendar`	`exchange_id`, `cal_date`, `is_open`	Drives every date arithmetic
`rating_scale`	`scale_id`, `firm_id`, `from_date`, `to_date`, `n_tiers`	Effective-dated

Table	Key columns	Notes
`rating_scale_map`	`scale_id`, `native_label`, `canonical_rating`, `rank_within_scale`	Native → canonical

Event (system of record, append-only)

`rating_event` is the core fact table.

Column	Type	Notes
`event_id`	PK	
`analyst_id`, `firm_id`, `security_id`	FK	
`announced_ts_utc`	TIMESTAMP	Minute precision, not date
`effective_ts_utc`	TIMESTAMP	May lag announcement
`knowledge_ts_utc`	TIMESTAMP	When the system learned it — enables bitemporal replay
`action_type`	TEXT	initiation, upgrade, downgrade, reiteration, pt_change, suspend, restrict, resume, drop
`native_rating`	TEXT	Preserved verbatim
`canonical_rating`	TEXT	strong_buy, buy, hold, sell, strong_sell, not_rated, under_review, restricted
`prior_canonical_rating`	TEXT	Nullable on initiation
`price_target`, `pt_currency`, `pt_horizon_months`		Nullable
`expiry_date`	DATE	Your expiry concept
`scale_id`	FK	Which scale version applied
`source`, `supersedes_event_id`, `is_correction`		Correction chain

Indexes: `(security_id, announced_ts_utc)`, `(analyst_id, announced_ts_utc)`, `(knowledge_ts_utc)`.

Market

Table	Key columns	Notes
`price_daily`	`security_id`, `price_date`, `open`, `high`, `low`, `close`, `volume`, `adj_factor`	Unadjusted raw + factor, so adjustments are reproducible
`total_return_daily`	`security_id`, `price_date`, `tr_index`	Dividend-reinvested index, base 100
`corporate_action`	`security_id`, `effective_date`, `action_type`, `ratio`, `cash_amount`	split, dividend, spinoff, rights, merger
`delisting_return`	`security_id`, `final_date`, `final_return`	−1.0 for bankruptcy; takeout price for M&A
`benchmark_series`	`benchmark_id`, `price_date`, `tr_index`	Market and sector indices in one table
`benchmark_def`	`benchmark_id`, `name`, `kind`, `rule_json`	kind ∈ market, sector, custom
`factor_return_daily`	`factor_id`, `price_date`, `ret`	mkt, smb, hml, mom
`security_beta`	`security_id`, `as_of_date`, `beta`, `window_days`	Rolling estimate
`fx_rate_daily`	`from_ccy`, `to_ccy`, `rate_date`, `rate`	
`liquidity_daily`	`security_id`, `price_date`, `adv_20d`, `free_float`	Implementability filters

Consensus

Table	Key columns	Notes
`consensus_daily`	`security_id`, `snap_date`, `mean_rating_score`, `n_ratings`, `pct_buy`, `median_pt`	Rebuilt from `rating_event`; needed for relative scoring

Derived (rebuilt per run)

Table	Key columns	Notes
`rating_interval`	`interval_id`, `event_id`, `analyst_id`, `security_id`, `entry_date`, `exit_date`, `exit_reason`, `position_weight`	exit_reason ∈ superseded, expired, dropped, delisted, analyst_left, run_end
`interval_return`	`interval_id`, `horizon`, `stock_ret`, `bench_ret`, `excess_ret`, `contribution`, `car_pm1`, `pre_event_ret_20d`	One row per horizon per interval
`analyst_metric`	`run_id`, `analyst_id`, `metric_code`, `value`, `n_obs`, `std_err`, `t_stat`, `ci_low`, `ci_high`, `shrunk_value`, `peer_percentile`	Long format — avoids schema churn as metrics are added
`analyst_composite`	`run_id`, `analyst_id`, `composite_score`, `rank_overall`, `rank_in_peer_group`, `is_ranked`	
`portfolio_position`	`run_id`, `analyst_id`, `position_date`, `security_id`, `weight`	Simulated book
`portfolio_nav`	`run_id`, `analyst_id`, `position_date`, `nav`, `bench_nav`, `drawdown`	Equity curve

Config and audit

Table	Key columns
`methodology_version`	`methodology_version_id`, `name`, `params_json`, `created_at`, `is_baseline`
`calc_run`	`run_id`, `methodology_version_id`, `as_of_date`, `universe_filter_json`, `started_at`, `finished_at`, `status`, `row_counts_json`
`scenario`	`scenario_id`, `name`, `base_run_id`, `methodology_version_id`, `notes`

SQLite specifics

- `PRAGMA journal_mode=WAL` and `foreign_keys=ON` at connection open.
- Daily price data for 500 securities × 10 years is roughly 1.25 M rows — comfortable, but index `(security_id, price_date)` and keep the table narrow.
- Use a date dimension or the `trading_calendar` table for all business-day offsets; never compute them in Python with `timedelta`.
- Consider `DuckDB` purely as a read-side query engine over the same file later if aggregation gets slow; do not design for it now.

Layer 2 — Synthetic data

The generator is also your test oracle. Plant known skill in known analysts, then check the engine recovers it. If it cannot, the engine is wrong — not the data.

Target scale

Entity	Count	Rationale
Securities	500	11 sectors, 3 cap buckets
Trading days	~2,520	10 years, 2016-01-01 to 2025-12-31
Firms	25	Mixed tiers and rating scales
Analysts	180	Avg 14 names each, long right tail
Rating events	~95,000	~3.8 changes per name per year

Generation order — each step depends only on earlier ones.

1. Calendar and exchanges. Weekdays minus a fixed holiday list.
2. Sectors, then securities assigned to sector and cap bucket. Assign each a true beta (0.6–1.6) and idiosyncratic vol (15–60% annualized, higher for small caps and biotech).
3. Factor returns. Daily market return as Student-t (df 4) with vol clustering from a simple GARCH(1,1) — fat tails matter, Gaussian prices make every risk metric look tame. Add size, value, momentum factor series.
4. Security returns: `r = beta·mkt + sector_loading·sector_factor + idio`. Sector factor gives you intra-sector correlation, which is what makes sector-relative benchmarking meaningful.
5. Corporate actions. Sprinkle splits (~2%/yr), quarterly dividends on ~60% of names, a handful of spinoffs. Build `price_daily` unadjusted, then `total_return_daily` as the dividend-reinvested index.

6. Delistings. ~1.5%/yr: bankruptcies with final return −1.0, takeouts at a 25–40% premium. **This is the survivorship test** — if your engine silently drops these, the bug only shows here.
7. Benchmarks: broad market index and one total-return index per sector, cap-weighted from the securities.
8. Firms, rating scales (mix of 3-tier and 5-tier, with one firm changing scale mid-history to exercise the effective-dated mapping), analysts, employment spells (~12% move firms), coverage spells with transfers.
9. Rating events — the interesting part, below.
10. Consensus snapshots, derived from the generated events.

Generating ratings with planted skill

Give each analyst a latent skill parameter `s ~ N(0, 0.35)` in annualized alpha terms, plus behavioral traits: `herding` (0–1), `momentum_chasing` (0–1), `optimism_bias`, `revision_frequency`, `pt_aggressiveness`.

For each (analyst, security, decision date):

- Peek at the security's *future* excess return over the next 126 days — call it `f`.
- Form a signal `signal = s·f + noise`, where noise variance is set so the analyst's information coefficient equals their intended skill. A skill-zero analyst gets pure noise; a skilled one gets a genuinely predictive signal.
- Blend toward consensus by `herding`, and toward trailing 20-day return by `momentum_chasing`.
- Map the blended score to a canonical rating through thresholds skewed bullish: roughly 55% buy-side, 38% hold, 7% sell. This reproduces the real-world skew that makes absolute scoring misleading.
- Price target = current price × (1 + implied upside), with implied upside drawn around the rating's central tendency, scaled by `pt_aggressiveness` and the security's volatility.
- Set `expiry_date` at 9–15 months out. Let 10–15% of ratings lapse without action so the expiry policy has something to bite on.

Reserved test fixtures — hardcode a handful of analysts with exact properties, outside the random draw:

Fixture	Property	Engine must show
`A_ORACLE`	Perfect foresight, always right	Top rank, IC near 1.0
`A_INVERSE`	Perfectly wrong	Bottom rank, negative alpha

Fixture	Property	Engine must show
`A_NOISE`	Pure coin flip, 400 ratings	Alpha ≈ 0, t-stat < 2
`A_LUCKY`	Coin flip, only 6 ratings, all winners	High raw score, **not ranked**, heavily shrunk
`A_HERDER`	Always exactly consensus	Herding score ≈ 0, no relative alpha
`A_SLEEPER`	30 ratings, never updated, all expired	Stale coverage ≈ 100%
`A_BANKRUPT`	Covers a name that goes to zero	Interval closes at −100%, not dropped

Implementation notes

- Seeded RNG, seed stored in `calc_run` and in a `generation_manifest` row. Same seed must produce a byte-identical database.
- Generate with NumPy vectorized over the full date grid, then bulk-insert via `executemany` inside one transaction. Row-at-a-time inserts will take minutes instead of seconds.
- Write a `--scale small|medium|full` flag. The small profile (50 securities, 2 years, 20 analysts) is what you develop against; full is for performance work.
- Introduce deliberate dirt in ~1% of rows: a duplicate rating event, a ticker that changes mid-history, a rating referencing a delisted security. The ingestion validator must catch these, and you want them present from day one rather than discovered in production.

Layer 3 — Calculation engine

Four stages, each a pure function of (inputs, methodology). No stage reads global state or the clock. This is what makes what-if analysis cheap later: a scenario is just the same pipeline with a different config.

Stage 1 — Interval builder (`engine/intervals.py`)

Input: rating events for one (analyst, security). Output: non-overlapping intervals with a position weight.

1. Sort by `announced_ts_utc`, resolve correction chains (`supersedes_event_id`), drop superseded rows.
2. Normalize `native_rating` → `canonical_rating` using the scale effective on that date.

3. Entry date = resolved from `entry_convention` against the trading calendar. Pre-market announcement → same-day open is defensible; post-close → next open.
4. Exit date = earliest of: next event from same analyst on same security; `expiry_date` under `hard_close`; coverage spell end; employment spell end; delisting date; run as-of date.
5. Non-investable ratings (`not_rated`, `restricted`, `under_review`) produce an interval with `position_weight = NULL` and are excluded from return math — never coerced to hold.
6. Position weight from `position_weights`, or from the consensus z-score when `scoring_mode = relative_to_consensus`.

Stage 2 — Return engine (`engine/returns.py`)

For each interval and each horizon (to-exit, plus every entry in `fixed_horizons_days`):

```
ER_i \;=\; \frac{TR_{i}(t_1)}{TR_{i}(t_0)} \;-\; \frac{B_{i}(t_1)}{B_{i}(t_0)}
```

where `TR` is the security's total-return index and `B` the benchmark index selected by `benchmark_mode`. Under `beta_adjusted`, the benchmark leg becomes `1 + β·(market return)`. Contribution is `C_i = w_i · ER_i`.

Also computed per interval: announcement CAR over [−1, +1], pre-event return over the prior 20 days, and realized return versus the price target's implied upside.

Edge cases that must be handled explicitly, not by accident: delisting mid-interval (use `delisting_return`, terminate), halted trading (carry last price, flag), interval shorter than one trading day (drop, flag), missing benchmark data (fail the run, do not substitute zero).

Stage 3 — Metric engine (`engine/metrics.py`)

Each metric is a registered function with a `metric_code`, a required-inputs declaration and a scalar output. Results land in `analyst_metric` in long format. Register the five families from the analysis: return/skill, timing, price-target, behavior, consistency. A new metric is a new function plus a registry entry — no schema change.

The portfolio simulation lives here too: equal-weight long the positive weights, short the negative, rebalance on each event date, benchmark-neutral, costs applied at `transaction_cost_bps` per unit of turnover. It produces `portfolio_position` and `portfolio_nav`, from which annualized alpha, tracking error, information ratio and max drawdown all fall out.

Stage 4 — Statistics layer (`engine/stats.py`)

- Standard error and t-statistic on mean contribution. Block bootstrap (1,000 draws, blocks by calendar quarter) for confidence intervals — ratings cluster in time, so i.i.d. bootstrap understates the interval.
- Empirical Bayes shrinkage toward the peer-group mean:

```
\hat{\alpha}_a^{\,shrunk} \;=\; \lambda_a\,\hat{\alpha}_a \;+\; (1-\lambda_a)\,\bar{\alpha}_{peer}, \qquad \lambda_a \;=\; \frac{\tau^2}{\tau^2 + \sigma_a^2/n_a}
```

- Peer normalization: z-score within `peer_group_dims`, then percentile. Composite score = weighted sum of standardized family scores, mapped to 0–100.
- `is_ranked = false` whenever `n_obs < min_ratings_for_rank`. Scores still compute and display; the analyst just does not appear on the leaderboard.

Run model

A run is a row in `calc_run` plus the derived rows tagged with its `run_id`. Runs are immutable and comparable. Stage outputs are cached on disk as Parquet keyed by `(stage, methodology_hash, universe_hash)`, so a what-if that only changes `position_weights` reuses the interval and return stages and recomputes from Stage 3 — seconds instead of minutes. Build the hashing in from the start; bolting it on later means touching every stage.

Layer 4 — FastAPI backend

The API is a thin layer over `engine/`. No calculation logic lives in a router. Every response that carries a number also carries the `run_id` and `methodology_version_id` that produced it, so the UI can always label what it is showing.

Route map

Method	Path	Returns
GET	`/analysts`	Paged list, filterable by sector, firm, region, cap bucket, `is_ranked`
GET	`/analysts/{id}`	Profile, coverage history, employment
GET	`/analysts/{id}/metrics`	All metric values with n, t-stat, CI, shrunk value, percentile
GET	`/analysts/{id}/ratings`	Every rating with its interval, return, benchmark and contribution — the drill-down

Method	Path	Returns
GET	`/analysts/{id}/portfolio`	NAV series, positions, drawdown
GET	`/leaderboard`	Ranked table for a run, with peer-group filters
GET	`/securities`, `/securities/{id}`	Reference, price history, consensus history
GET	`/securities/{id}/ratings`	All analysts covering it, over time
POST	`/methodologies`	Create a methodology version from a params payload
GET	`/methodologies`, `/methodologies/{id}`	List, fetch, diff against baseline
POST	`/runs`	Start a calculation run → `{run_id, status}`
GET	`/runs/{id}`	Status, progress, row counts, timings
GET	`/runs/compare?a=&b=`	Metric-level diff between two runs
POST	`/scenarios`	Define a what-if from a base run plus parameter overrides
POST	`/scenarios/{id}/execute`	Run it
GET	`/scenarios/{id}/diff`	Per-analyst delta in score and rank
POST	`/benchmarks/custom`	Define a custom benchmark from a rule
POST	`/backtest`	Configure and run a portfolio simulation
GET	`/backtest/{id}`	Equity curve, stats, positions, trade log
GET	`/meta/enums`	Canonical ratings, exit reasons, metric codes — so the UI never hardcodes them

Contracts

Pydantic models in `api/schemas/`, one module per resource. Three conventions worth fixing early:

- Every list endpoint takes `limit`, `offset`, `sort_by`, `sort_dir` and returns `{items, total, limit, offset}`.

- Every metric value serializes as an object, not a float: `{value, n_obs, t_stat, ci_low, ci_high, shrunk_value, peer_percentile, is_significant}`. The UI should never have to decide whether a number is trustworthy; the API already said.
- Errors return `{code, message, detail}` with a stable `code` string, so Streamlit can branch on it.

Long-running work

A full run over 180 analysts and 95,000 ratings will exceed a comfortable HTTP timeout. Use FastAPI `BackgroundTasks` with run state in the `calc_run` table, and have the UI poll `GET /runs/{id}`. Avoid Celery or a broker — it is a significant operational cost for a single-node app, and the polling pattern is enough. Guard against concurrent writes with a single-writer discipline: one run executes at a time, enforced by a status check plus SQLite's write lock.

Caching

`@lru_cache` on reference lookups (calendar, sectors, scale maps). HTTP `ETag` on leaderboard and metric responses keyed by `run_id`, since a run's results never change once written.

Layer 5 — Streamlit UI

Eight pages. Streamlit reruns the whole script on every interaction, so the discipline is: all data access goes through `api_client.py`, every call wrapped in `@st.cache_data` keyed on `run_id` plus the filter arguments. Filters live in `st.session_state` so they survive navigation.

Page	Purpose	Key interactions
Home / Run control	Pick the active run, start a new one, watch progress	Methodology picker, as-of date, universe filter, progress poll
Leaderboard	Ranked analysts for the active run	Peer-group filters, rank-eligibility toggle, CI error bars on the score column, click-through to profile
Analyst profile	Everything about one analyst	Metric cards by family, equity curve vs. benchmark, rating history table, performance by sector and regime
Rating drill-down	The audit trail for a single call	Entry/exit prices, benchmark leg, excess return, the arithmetic shown step by step

Page	Purpose	Key interactions
Security view	All coverage of one name	Price chart with rating changes marked, consensus band, who called it right
Methodology lab	Create and compare methodology versions	Parameter form, diff against baseline, execute button
What-if	Scenario builder and comparison	Toggle panel, side-by-side ranks, rank-change waterfall
Backtest sandbox	Portfolio simulation	Configuration form, equity curve, stats table, trade log export

Workflow — the path a user actually walks

1. Land on Home, see the baseline run already selected and its as-of date.
2. Open Leaderboard, filter to Technology / large cap. See 23 ranked analysts and a note that 11 more are unranked for insufficient sample.
3. Click an analyst. Profile shows composite 72, annualized alpha 4.1% with a 95% CI of [0.8%, 7.3%] and n = 41 — the band is as prominent as the point estimate.
4. Click a single rating in the history table. Drill-down shows: upgraded to Buy on 2024-03-14 pre-market, entered at the 03-15 open at 142.30, exited 2024-09-02 at 171.80 on a downgrade, stock +22.1%, sector benchmark +9.4%, excess +12.7%, weight +0.5, contribution +6.35%.
5. Open Methodology lab, clone the baseline, switch `benchmark_mode` from sector to market, run it.
6. Open What-if, compare the two runs. See which analysts moved and by how much — sector specialists in a hot sector jump, which is exactly the artifact a user needs in order to understand why sector-relative is the default.

UI conventions worth fixing early

- Never show a score without its sample size. A metric card displays `4.1% (n=41, t=2.3)`, not `4.1%`.
- Unranked analysts render greyed with a one-line reason, not hidden. Hiding them invites the question of where they went in every demo.
- Every table has a CSV download. Analysts will want to check your arithmetic in Excel, and letting them do so builds trust faster than any explanation.
- The active `run_id` and methodology name sit in the sidebar on every page. The most common failure mode in a system like this is a user comparing two numbers computed under different rules without realizing it.

- Plotly for charts — hover and zoom matter on a 10-year equity curve. Keep it to one chart library.

Layer 6 — What-if and methodology toggles

A scenario is a base run plus a set of parameter overrides. Because the engine is a pure function of config, this layer is mostly plumbing and presentation — which is the payoff for having built the methodology object in Layer 0.

Scenario lifecycle

1. User clones the baseline methodology in the UI and changes one or more toggles.
2. The API creates a new `methodology_version` row and a `scenario` row linking it to the base run.
3. Execute. The runner hashes the config, finds which pipeline stages are unaffected, reuses their cached outputs and recomputes from the first affected stage.
4. The diff endpoint joins `analyst_metric` across the two runs and returns per-analyst deltas.

Which toggles invalidate which stage — this table drives the cache logic and is worth getting right once.

Toggle	First stage to recompute
`entry_convention`	Stage 1 — intervals
`expiry_policy`, `decay_days`	Stage 1 — intervals
`benchmark_mode`, `benchmark_id`	Stage 2 — returns
`return_basis`	Stage 2 — returns
`fixed_horizons_days`	Stage 2 — returns
`position_weights`	Stage 2 — contribution only
`scoring_mode`	Stage 1 — weights are consensus-derived
`transaction_cost_bps`	Stage 3 — simulation only
`min_ratings_for_rank`	Stage 4 — stats only
`shrinkage_method`, `peer_group_dims`	Stage 4 — stats only

The common cases — changing the rank threshold or the shrinkage method — touch only Stage 4 and return in under a second, which is what makes the toggle panel feel interactive rather than like a batch job.

Comparison views

- **Rank migration**: a slope chart, baseline rank on the left, scenario rank on the right, lines connecting. The analysts whose lines cross steeply are the story.
- **Delta table**: analyst, baseline score, scenario score, delta, rank delta, sorted by absolute rank change.
- **Sensitivity sweep**: hold everything fixed, vary one parameter across a range, plot each analyst's score as a line. An analyst whose rank is stable across every plausible methodology is genuinely good; one who is top-decile only under a single parameter setting is an artifact.
- **Agreement metric**: Spearman correlation between the two rankings, reported as one number. If two defensible methodologies produce a rank correlation of 0.4, the honest conclusion is that the system cannot distinguish these analysts, and the UI should say so rather than present one ordering as the answer.

Guardrails

Scenarios are saved, named and shareable by `scenario_id`, so a result shown in a meeting can be reproduced exactly. Mark one methodology version `is_baseline = true`; everything else is explicitly a deviation. Never let a scenario silently become the default run — the leaderboard always names which methodology it is showing.

Layer 7 — Backtest sandbox and custom benchmarks

This is where the system stops being a scorecard and becomes something a portfolio manager will use: *if I had followed this set of analysts under these rules, what would have happened?*

Backtest configuration

Parameter	Options
Analyst selection	Named list, top-N by composite, top quartile in a peer group, all
Signal rule	Follow upgrades only, follow all Buy-or-better, follow consensus-relative deviations
Position sizing	Equal weight, conviction weight, inverse volatility, score weight
Rebalance	On each rating event, weekly, monthly
Long/short	Long only, long-short, market neutral, beta neutral

Parameter	Options
Constraints	Max position %, max sector %, min ADV multiple, exclude unshortable
Costs	Bps per trade, optional slippage model
Period	Any start/end inside the data range

Outputs

Equity curve against benchmark; annualized return, volatility, Sharpe, information ratio, max drawdown, turnover, hit rate; monthly return heatmap; attribution by sector, by analyst and by rating category; a full trade log the user can export.

Point-in-time discipline — the thing that makes a backtest honest

Every selection decision uses only data available on the decision date. Selecting the top-10 analysts by full-period composite and then backtesting them over that same period produces a spectacular and completely meaningless curve. Implement selection as a rolling exercise: at each rebalance date, rank analysts on their trailing performance *as of that date only*, then follow the resulting set forward. Build this constraint into the API signature so it cannot be bypassed — the endpoint takes a ranking lookback window, not a precomputed analyst list, when selection is dynamic.

Use `knowledge_ts_utc`, not `announced_ts_utc`, for availability. A rating that was corrected three weeks later was not actionable in its corrected form on day one.

Custom benchmarks

A custom benchmark is a rule stored in `benchmark_def.rule_json` and materialized into `benchmark_series` on creation:

- A filter over the security universe (sector, cap bucket, region, liquidity).
- A weighting method (equal, cap, float-adjusted).
- A rebalance frequency.

Once materialized it is selectable anywhere `benchmark_id` appears, which means the same object serves the metric engine, the what-if layer and the backtest. Offer a few presets — sector-neutral, cap-matched, equal-weighted market — since those cover most of what people actually want and spare them the rule builder.

Other self-service worth including

- **Peer-group builder**: let users define a cohort by any combination of dimensions and get a leaderboard restricted to it.

- **Metric weight editor**: adjust the composite's family weights and watch the ranking reorder live. Being able to see that the top of the leaderboard is stable under reweighting is more convincing than any defense of a particular weight set.
- **Coverage gap view**: securities with thin or no analyst coverage. Not a performance metric, but the first thing a research director asks for.
- **Saved views**: filter state plus run id, bookmarkable.

Layer 8 — Testing and hardening

Golden fixtures

A tiny hand-built dataset — 3 analysts, 5 securities, 20 ratings, 60 trading days — where every expected number is computed by hand in a spreadsheet and checked into the repo. Every engine change runs against it first. When a number moves, either the change was intended and the fixture updates, or you just found a regression. Without this, methodology changes become unverifiable.

Property tests

Invariants that must hold for any input, checked with Hypothesis:

- Intervals for one (analyst, security) never overlap and never have gaps that are silently treated as holds.
- Sum of interval durations equals total coverage duration minus non-investable spells.
- Excess return is zero when the stock and its benchmark are identical series.
- Reversing every rating sign negates every contribution exactly.
- Shrinkage never moves an estimate past the peer mean.
- Rankings are invariant to row insertion order in the database.

Oracle tests

Run the full engine against the synthetic data and assert the planted fixtures come out as specified in Layer 2 — oracle at the top, inverse at the bottom, noise insignificant, lucky analyst unranked, bankrupt-stock interval closing at −100%. This is the end-to-end check that the pipeline composes correctly.

Performance targets

Operation	Target	Approach if missed
Full run, 180 analysts, 95k ratings	< 90 s	Vectorize returns in NumPy; batch the interval builder by security
Stage-4-only what-if	< 2 s	Already cached; check the join

Operation	Target	Approach if missed
Leaderboard query	< 300 ms	Index `(run_id, metric_code)`; materialize the composite
Analyst profile load	< 500 ms	Single joined query, not N+1 per metric
Backtest, 10 years	< 30 s	Precompute the return matrix once per run

Hardening

- Input validation on every ingestion path: rating against a known scale, security exists and was listed on that date, timestamps monotonic within a correction chain, no duplicate `(analyst, security, announced_ts)`.
- Structured logging with `run_id` on every line, so a bad number can be traced to the run that made it.
- `GET /health` reporting DB connectivity, row counts and last successful run.
- Reproducibility test in CI: run the generator twice with the same seed, assert identical checksums over every table.
- Packaging: `docker-compose` with two services (API, UI) and a volume for the SQLite file. Makefile targets `make seed`, `make run`, `make test`.

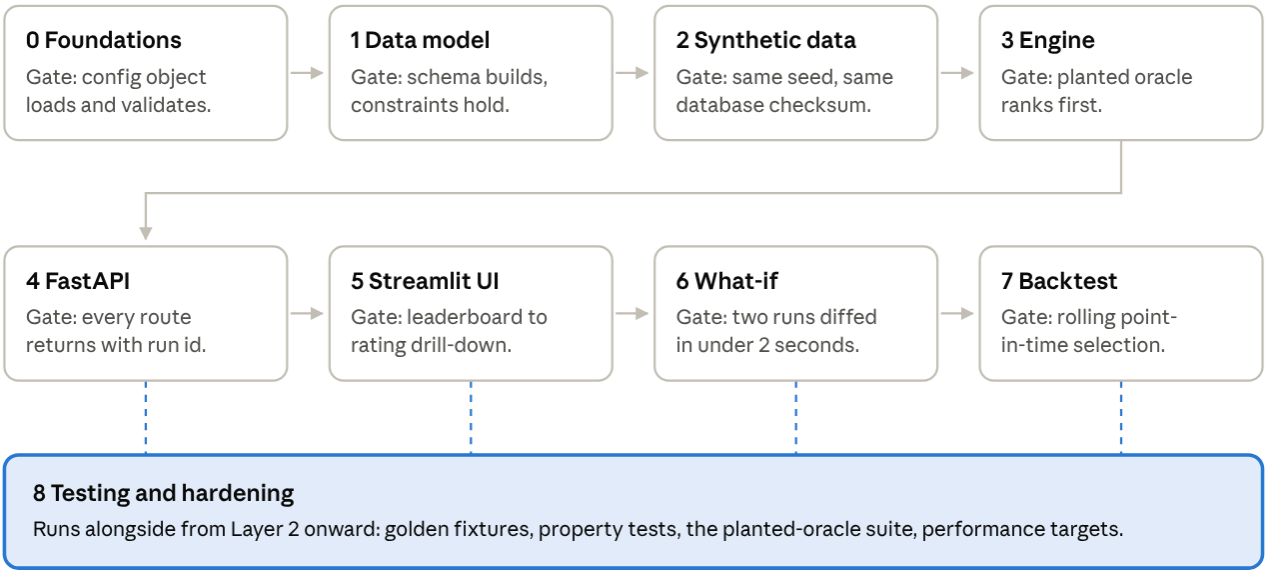
Known limitation to document in the app itself

The system measures rating skill only. It does not measure client service, model quality, corporate access or commission generation — which is how analysts are actually compensated. Put that sentence on the leaderboard page. It costs nothing and prevents the system from being quietly misused in compensation discussions, which is the most likely way a tool like this causes harm.

Build sequence

Each layer ships with a gate that must pass before the next begins

Eight build layers, left to right, then down



build sequence · 8 layers, 1 cross-cutting

The order is not negotiable in one place: Layers 0 to 3 must be finished and passing their gates before any UI work starts. The temptation will be to build a Streamlit screen early because it demos well — resist it, because every screen you build against an unfinished engine gets rewritten.

Two useful checkpoints. After Layer 3 you have a command-line tool that prints a correct leaderboard from synthetic data; that is the real proof the system works, and it arrives before anything is visible. After Layer 5 you have a complete product that a stakeholder can use, and Layers 6 to 8 are value added on a working base.

If the timeline compresses, cut from Layer 7 first — the backtest sandbox is the most impressive and the least necessary. Never cut Layer 8's golden fixtures and planted-oracle tests; without them you cannot tell a methodology change from a bug, and that distinction is the whole value of the system.